

Properties of Regular Languages

Closure, Decision Algorithms, and the Pumping Lemma

Computer Science Theory

Slide Set 4

Table of Contents

- 1 Closure Properties
- 2 Decision Algorithms
- 3 The Pumping Lemma
- 4 Proving Non-Regularity
- 5 Review and Exercises

By the end of this slide set, you should be able to:

- prove closure of the regular languages using automaton constructions and set identities;
- decide membership, emptiness, finiteness, equality, containment, and universality for regular languages;
- state and explain the quantifier structure of the pumping lemma;
- use the pumping lemma to prove selected languages nonregular; and
- recognize invalid pumping-lemma arguments and repair them.

Part 1: Core Closure Properties

Self-Study Header: Operating within the Regular World

A class of languages is **closed** under an operation if applying that operation to members of the class always results in a member of the same class.

Closure Theorem

The family of regular languages is closed under union, intersection, concatenation, complement, and Kleene star.

Formally, if L_1 and L_2 are regular languages, the following are also regular:

- 1 **Union:** $L_1 \cup L_2$
- 2 **Concatenation:** $L_1 L_2$
- 3 **Kleene star:** L_1^*
- 4 **Complement:** $\overline{L_1} = \Sigma^* \setminus L_1$
- 5 **Intersection:** $L_1 \cap L_2$

Part 1: Core Closure Properties

Self-Study Header: Operating within the Regular World

A class of languages is **closed** under an operation if applying that operation to members of the class always results in a member of the same class.

Closure Theorem

The family of regular languages is closed under union, intersection, concatenation, complement, and Kleene star.

Formally, if L_1 and L_2 are regular languages, the following are also regular:

- 1 **Union:** $L_1 \cup L_2$
- 2 **Concatenation:** $L_1 L_2$
- 3 **Kleene star:** L_1^*
- 4 **Complement:** $\overline{L_1} = \Sigma^* \setminus L_1$
- 5 **Intersection:** $L_1 \cap L_2$

Part 1: Core Closure Properties

Self-Study Header: Operating within the Regular World

A class of languages is **closed** under an operation if applying that operation to members of the class always results in a member of the same class.

Closure Theorem

The family of regular languages is closed under union, intersection, concatenation, complement, and Kleene star.

Formally, if L_1 and L_2 are regular languages, the following are also regular:

- 1 **Union:** $L_1 \cup L_2$
- 2 **Concatenation:** $L_1 L_2$
- 3 **Kleene star:** L_1^*
- 4 **Complement:** $\overline{L_1} = \Sigma^* \setminus L_1$
- 5 **Intersection:** $L_1 \cap L_2$

Proof by Construction: Union Using Grammars

Theorem: If L_1 and L_2 are regular, then $L_1 \cup L_2$ is regular.

Regular-Grammar Construction

Let $G_1 = (V_1, \Sigma, S_1, P_1)$ and $G_2 = (V_2, \Sigma, S_2, P_2)$ be right-linear grammars for L_1 and L_2 . Rename variables so that $V_1 \cap V_2 = \emptyset$, and introduce a fresh start variable S . If unit productions are allowed, define

$$G = (V_1 \cup V_2 \cup \{S\}, \Sigma, S, P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}).$$

Correctness

Every derivation begins by choosing $S \rightarrow S_1$ or $S \rightarrow S_2$ and then follows productions from the selected grammar. Therefore $L(G) = L(G_1) \cup L(G_2) = L_1 \cup L_2$.

Proof by Construction: Union Using Grammars

Theorem: If L_1 and L_2 are regular, then $L_1 \cup L_2$ is regular.

Regular-Grammar Construction

Let $G_1 = (V_1, \Sigma, S_1, P_1)$ and $G_2 = (V_2, \Sigma, S_2, P_2)$ be right-linear grammars for L_1 and L_2 . Rename variables so that $V_1 \cap V_2 = \emptyset$, and introduce a fresh start variable S . If unit productions are allowed, define

$$G = (V_1 \cup V_2 \cup \{S\}, \Sigma, S, P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}).$$

Correctness

Every derivation begins by choosing $S \rightarrow S_1$ or $S \rightarrow S_2$ and then follows productions from the selected grammar. Therefore $L(G) = L(G_1) \cup L(G_2) = L_1 \cup L_2$.

Proof by Construction: Union Using Grammars

Theorem: If L_1 and L_2 are regular, then $L_1 \cup L_2$ is regular.

Regular-Grammar Construction

Let $G_1 = (V_1, \Sigma, S_1, P_1)$ and $G_2 = (V_2, \Sigma, S_2, P_2)$ be right-linear grammars for L_1 and L_2 . Rename variables so that $V_1 \cap V_2 = \emptyset$, and introduce a fresh start variable S . If unit productions are allowed, define

$$G = (V_1 \cup V_2 \cup \{S\}, \Sigma, S, P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}).$$

Correctness

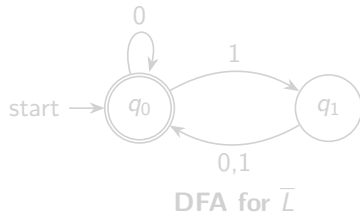
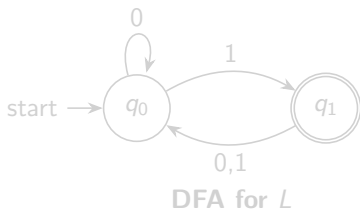
Every derivation begins by choosing $S \rightarrow S_1$ or $S \rightarrow S_2$ and then follows productions from the selected grammar. Therefore $L(G) = L(G_1) \cup L(G_2) = L_1 \cup L_2$.

Proof by Construction: Complementation

Theorem: If L is regular, then $\bar{L}$ is regular.

The Construction

Because L is regular, a *complete* DFA $M = (Q, \Sigma, \delta, q_0, F)$ recognizes L . Construct $M' = (Q, \Sigma, \delta, q_0, Q \setminus F)$ by swapping the accepting and nonaccepting states.



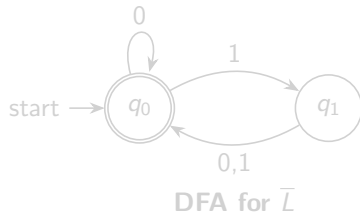
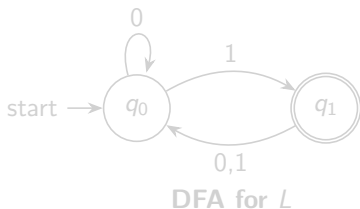
Crucial detail: Complete the DFA first by adding a sink state if necessary. Merely swapping final states in an NFA does not compute the complement, because acceptance is existential over its possible paths.

Proof by Construction: Complementation

Theorem: If L is regular, then $\bar{L}$ is regular.

The Construction

Because L is regular, a *complete* DFA $M = (Q, \Sigma, \delta, q_0, F)$ recognizes L . Construct $M' = (Q, \Sigma, \delta, q_0, Q \setminus F)$ by swapping the accepting and nonaccepting states.



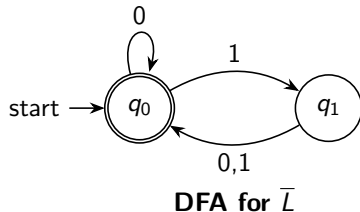
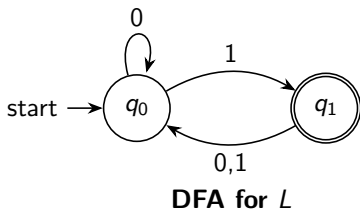
Crucial detail: Complete the DFA first by adding a sink state if necessary. Merely swapping final states in an NFA does not compute the complement, because acceptance is existential over its possible paths.

Proof by Construction: Complementation

Theorem: If L is regular, then $\bar{L}$ is regular.

The Construction

Because L is regular, a *complete* DFA $M = (Q, \Sigma, \delta, q_0, F)$ recognizes L . Construct $M' = (Q, \Sigma, \delta, q_0, Q \setminus F)$ by swapping the accepting and nonaccepting states.



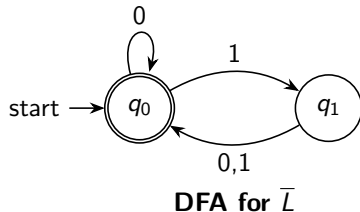
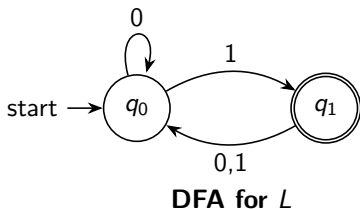
Crucial detail: Complete the DFA first by adding a sink state if necessary. Merely swapping final states in an NFA does not compute the complement, because acceptance is existential over its possible paths.

Proof by Construction: Complementation

Theorem: If L is regular, then $\bar{L}$ is regular.

The Construction

Because L is regular, a *complete* DFA $M = (Q, \Sigma, \delta, q_0, F)$ recognizes L . Construct $M' = (Q, \Sigma, \delta, q_0, Q \setminus F)$ by swapping the accepting and nonaccepting states.



Crucial detail: Complete the DFA first by adding a sink state if necessary. Merely swapping final states in an NFA does not compute the complement, because acceptance is existential over its possible paths.

Proof by Construction: Intersection

Theorem: If L_1 and L_2 are regular, then $L_1 \cap L_2$ is regular.

Cartesian Product Construction

Let $M_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$. We build a new DFA $M_\cap = (Q, \Sigma, \delta, q_0, F)$ that simulates both machines simultaneously:

- $Q = Q_1 \times Q_2$ (the states are pairs).
- $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$.
- $q_0 = (s_1, s_2)$.
- $F = F_1 \times F_2$ (accept iff **both** machines accept).

Alternative Proof via De Morgan's Law

Alternatively, $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$. Since regular languages are closed under union and complementation, they must be closed under intersection!

Proof by Construction: Intersection

Theorem: If L_1 and L_2 are regular, then $L_1 \cap L_2$ is regular.

Cartesian Product Construction

Let $M_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$. We build a new DFA $M_\cap = (Q, \Sigma, \delta, q_0, F)$ that simulates both machines simultaneously:

- $Q = Q_1 \times Q_2$ (the states are pairs).
- $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$.
- $q_0 = (s_1, s_2)$.
- $F = F_1 \times F_2$ (accept iff **both** machines accept).

Alternative Proof via De Morgan's Law

Alternatively, $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$. Since regular languages are closed under union and complementation, they must be closed under intersection!

Proof by Construction: Intersection

Theorem: If L_1 and L_2 are regular, then $L_1 \cap L_2$ is regular.

Cartesian Product Construction

Let $M_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$. We build a new DFA $M_\cap = (Q, \Sigma, \delta, q_0, F)$ that simulates both machines simultaneously:

- $Q = Q_1 \times Q_2$ (the states are pairs).
- $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$.
- $q_0 = (s_1, s_2)$.
- $F = F_1 \times F_2$ (accept iff **both** machines accept).

Alternative Proof via De Morgan's Law

Alternatively, $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$. Since regular languages are closed under union and complementation, they must be closed under intersection!

Regular languages are robust and closed under many other operations.

- **Reversal (L^R):** Reverse every transition, make the old start state the sole accept state, and add a new start state with ϵ -transitions to all old accept states.
- **Difference ($L_1 \setminus L_2$):** Since $L_1 \setminus L_2 = L_1 \cap \overline{L_2}$, and regular languages are closed under intersection and complement, they are closed under difference.
- **Homomorphism ($h(L)$):** For each transition labeled a , replace the edge by a path spelling $h(a)$; use an ϵ -transition when $h(a) = \epsilon$.
- **Right quotient:** $L_1/L_2 = \{x : \text{some } y \in L_2 \text{ satisfies } xy \in L_1\}$. If L_1 and L_2 are regular, then L_1/L_2 is regular.

Advanced Operations

Regular languages are robust and closed under many other operations.

- **Reversal (L^R):** Reverse every transition, make the old start state the sole accept state, and add a new start state with ε -transitions to all old accept states.
- **Difference ($L_1 \setminus L_2$):** Since $L_1 \setminus L_2 = L_1 \cap \overline{L_2}$, and regular languages are closed under intersection and complement, they are closed under difference.
- **Homomorphism ($h(L)$):** For each transition labeled a , replace the edge by a path spelling $h(a)$; use an ε -transition when $h(a) = \varepsilon$.
- **Right quotient:** $L_1/L_2 = \{x : \text{some } y \in L_2 \text{ satisfies } xy \in L_1\}$. If L_1 and L_2 are regular, then L_1/L_2 is regular.

Regular languages are robust and closed under many other operations.

- **Reversal (L^R):** Reverse every transition, make the old start state the sole accept state, and add a new start state with ε -transitions to all old accept states.
- **Difference ($L_1 \setminus L_2$):** Since $L_1 \setminus L_2 = L_1 \cap \overline{L_2}$, and regular languages are closed under intersection and complement, they are closed under difference.
- **Homomorphism ($h(L)$):** For each transition labeled a , replace the edge by a path spelling $h(a)$; use an ε -transition when $h(a) = \varepsilon$.
- **Right quotient:** $L_1/L_2 = \{x : \text{some } y \in L_2 \text{ satisfies } xy \in L_1\}$. If L_1 and L_2 are regular, then L_1/L_2 is regular.

Regular languages are robust and closed under many other operations.

- **Reversal (L^R):** Reverse every transition, make the old start state the sole accept state, and add a new start state with ε -transitions to all old accept states.
- **Difference ($L_1 \setminus L_2$):** Since $L_1 \setminus L_2 = L_1 \cap \overline{L_2}$, and regular languages are closed under intersection and complement, they are closed under difference.
- **Homomorphism ($h(L)$):** For each transition labeled a , replace the edge by a path spelling $h(a)$; use an ε -transition when $h(a) = \varepsilon$.
- **Right quotient:** $L_1/L_2 = \{x : \text{some } y \in L_2 \text{ satisfies } xy \in L_1\}$. If L_1 and L_2 are regular, then L_1/L_2 is regular.

Regular languages are robust and closed under many other operations.

- **Reversal (L^R):** Reverse every transition, make the old start state the sole accept state, and add a new start state with ε -transitions to all old accept states.
- **Difference ($L_1 \setminus L_2$):** Since $L_1 \setminus L_2 = L_1 \cap \overline{L_2}$, and regular languages are closed under intersection and complement, they are closed under difference.
- **Homomorphism ($h(L)$):** For each transition labeled a , replace the edge by a path spelling $h(a)$; use an ε -transition when $h(a) = \varepsilon$.
- **Right quotient:** $L_1/L_2 = \{x : \text{some } y \in L_2 \text{ satisfies } xy \in L_1\}$. If L_1 and L_2 are regular, then L_1/L_2 is regular.

Closure Proof Toolbox

Choose the representation that makes the operation easiest.

Automaton Constructions

Union/intersection: Use a product DFA; change only the accepting pairs ($\vee$ for union, $\wedge$ for intersection).

Complement: Complete a DFA and replace F by $Q \setminus F$.

Concatenation/star: Connect ε -NFAs using the same wrappers as in Thompson's construction.

Reversal: Reverse the transitions and exchange the roles of the initial and final boundaries.

Homomorphism: Replace each symbol edge by a path spelling its image.

Set Identities Save Work

Difference, symmetric difference, and De Morgan's laws reduce new closure questions to operations already known to preserve regularity.

Closure Proof Toolbox

Choose the representation that makes the operation easiest.

Automaton Constructions

Union/intersection: Use a product DFA; change only the accepting pairs ($\vee$ for union, $\wedge$ for intersection).

Complement: Complete a DFA and replace F by $Q \setminus F$.

Concatenation/star: Connect ε -NFAs using the same wrappers as in Thompson's construction.

Reversal: Reverse the transitions and exchange the roles of the initial and final boundaries.

Homomorphism: Replace each symbol edge by a path spelling its image.

Set Identities Save Work

Difference, symmetric difference, and De Morgan's laws reduce new closure questions to operations already known to preserve regularity.

Part 2: Elementary Questions

Self-Study Header: Algorithmic Decidability

Given a standard representation of a regular language L (DFA, NFA, regular expression, or regular grammar), can we algorithmically answer fundamental questions about it?

1. The Membership Question

Given a string w and a standard representation of L , is $w \in L$?

Algorithm: For a DFA, trace the unique path labeled w from q_0 and test whether it ends in F .

Running time: $O(|w|)$ for a supplied DFA. Converting an NFA, RE, or grammar to a DFA is preprocessing and may increase the representation size substantially.

Because the standard representations are effectively convertible, a DFA algorithm establishes decidability for all of them, though not necessarily with the same complexity.

Part 2: Elementary Questions

Self-Study Header: Algorithmic Decidability

Given a standard representation of a regular language L (DFA, NFA, regular expression, or regular grammar), can we algorithmically answer fundamental questions about it?

1. The Membership Question

Given a string w and a standard representation of L , is $w \in L$?

Algorithm: For a DFA, trace the unique path labeled w from q_0 and test whether it ends in F .

Running time: $O(|w|)$ for a supplied DFA. Converting an NFA, RE, or grammar to a DFA is preprocessing and may increase the representation size substantially.

Because the standard representations are effectively convertible, a DFA algorithm establishes decidability for all of them, though not necessarily with the same complexity.

Part 2: Elementary Questions

Self-Study Header: Algorithmic Decidability

Given a standard representation of a regular language L (DFA, NFA, regular expression, or regular grammar), can we algorithmically answer fundamental questions about it?

1. The Membership Question

Given a string w and a standard representation of L , is $w \in L$?

Algorithm: For a DFA, trace the unique path labeled w from q_0 and test whether it ends in F .

Running time: $O(|w|)$ for a supplied DFA. Converting an NFA, RE, or grammar to a DFA is preprocessing and may increase the representation size substantially.

Because the standard representations are effectively convertible, a DFA algorithm establishes decidability for all of them, though not necessarily with the same complexity.

2. The Emptiness Question

Given a regular language L , is $L = \emptyset$?

Algorithm: Run BFS or DFS from q_0 . The language is nonempty exactly when some state in F is reachable.

3. The Finiteness Question

Given a regular language L , is L finite or infinite?

Algorithm: Keep the *useful* states that are reachable from q_0 and can reach a final state. Then L is infinite exactly when the subgraph induced by these useful states contains a directed cycle.

2. The Emptiness Question

Given a regular language L , is $L = \emptyset$?

Algorithm: Run BFS or DFS from q_0 . The language is nonempty exactly when some state in F is reachable.

3. The Finiteness Question

Given a regular language L , is L finite or infinite?

Algorithm: Keep the *useful* states that are reachable from q_0 and can reach a final state. Then L is infinite exactly when the subgraph induced by these useful states contains a directed cycle.

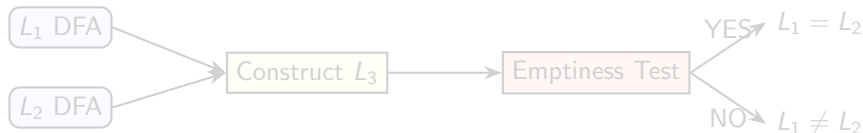
The Equality Algorithm

Question: Given two regular languages L_1 and L_2 , is $L_1 = L_2$?

We answer this using the **symmetric difference**:

$$L_3 = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2).$$

L_3 contains all strings that are in one language but not the other. $L_1 = L_2$ if and only if $L_3 = \emptyset$.



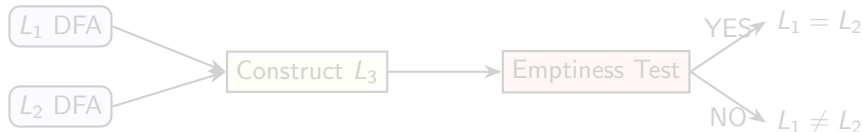
The Equality Algorithm

Question: Given two regular languages L_1 and L_2 , is $L_1 = L_2$?

We answer this using the **symmetric difference**:

$$L_3 = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2).$$

L_3 contains all strings that are in one language but not the other. $L_1 = L_2$ if and only if $L_3 = \emptyset$.



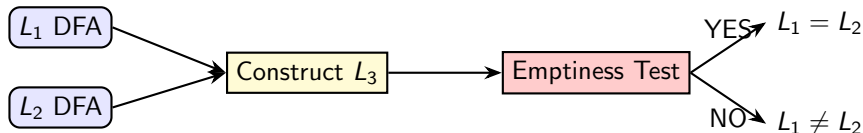
The Equality Algorithm

Question: Given two regular languages L_1 and L_2 , is $L_1 = L_2$?

We answer this using the **symmetric difference**:

$$L_3 = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2).$$

L_3 contains all strings that are in one language but not the other. $L_1 = L_2$ if and only if $L_3 = \emptyset$.



More Decidable Questions

Once emptiness and the product construction are available, several other questions follow immediately.

Reductions to Emptiness

$$L = \Sigma^* \iff \bar{L} = \emptyset$$

(universality),

$$L_1 \subseteq L_2 \iff L_1 \cap \bar{L}_2 = \emptyset$$

(containment),

$$L_1 \cap L_2 = \emptyset$$

(disjointness).

Direct Product Test

For equality, build the product DFA and search for a reachable pair in which exactly one component is accepting. Such a path labels a concrete counterexample string distinguishing the two languages.

More Decidable Questions

Once emptiness and the product construction are available, several other questions follow immediately.

Reductions to Emptiness

$$L = \Sigma^* \iff \bar{L} = \emptyset$$

(universality),

$$L_1 \subseteq L_2 \iff L_1 \cap \bar{L}_2 = \emptyset$$

(containment),

$$L_1 \cap L_2 = \emptyset$$

(disjointness).

Direct Product Test

For equality, build the product DFA and search for a reachable pair in which exactly one component is accepting. Such a path labels a concrete counterexample string distinguishing the two languages.

Part 3: Concept & The Pigeonhole Principle

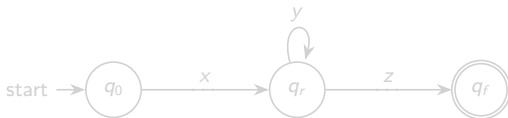
Self-Study Header: The Limits of Finite Memory

A DFA has finite memory: its current state records everything relevant about the prefix read so far. What happens when a computation is longer than the number of available states?

The Pigeonhole Principle

If p pigeons are placed into fewer than p holes, some hole has to have more than one pigeon in it.

If a DFA with n states reads a string of length at least n , consider the states visited before reading any symbol and after each of the first n symbols. Among these $n + 1$ visits, some state repeats. The intervening nonempty input labels a loop within the first n symbols.



Part 3: Concept & The Pigeonhole Principle

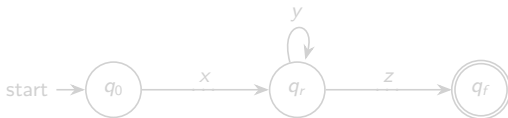
Self-Study Header: The Limits of Finite Memory

A DFA has finite memory: its current state records everything relevant about the prefix read so far. What happens when a computation is longer than the number of available states?

The Pigeonhole Principle

If p pigeons are placed into fewer than p holes, some hole has to have more than one pigeon in it.

If a DFA with n states reads a string of length at least n , consider the states visited before reading any symbol and after each of the first n symbols. Among these $n + 1$ visits, some state repeats. The intervening nonempty input labels a loop within the first n symbols.



Part 3: Concept & The Pigeonhole Principle

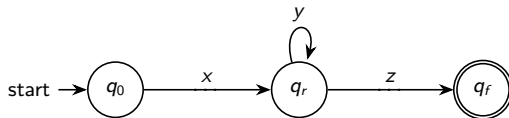
Self-Study Header: The Limits of Finite Memory

A DFA has finite memory: its current state records everything relevant about the prefix read so far. What happens when a computation is longer than the number of available states?

The Pigeonhole Principle

If p pigeons are placed into fewer than p holes, some hole has to have more than one pigeon in it.

If a DFA with n states reads a string of length at least n , consider the states visited before reading any symbol and after each of the first n symbols. Among these $n + 1$ visits, some state repeats. The intervening nonempty input labels a loop within the first n symbols.



Formal Statement: The Pumping Lemma

Pumping lemma for regular languages

If L is regular, then there exists $p \geq 1$ such that every $s \in L$ with $|s| \geq p$ can be written as $s = xyz$ satisfying:

- 1 $xy^iz \in L$ for every $i \geq 0$,
- 2 $|y| > 0$, and
- 3 $|xy| \leq p$.

Breaking it down:

- Condition 1: The same decomposition works for every pumping value $i \geq 0$.
- Condition 2: The loop consumes at least one symbol, so pumping actually changes the string when $i \neq 1$.
- Condition 3: The loop lies within the first p symbols.

Formal Statement: The Pumping Lemma

Pumping lemma for regular languages

If L is regular, then there exists $p \geq 1$ such that every $s \in L$ with $|s| \geq p$ can be written as $s = xyz$ satisfying:

- 1 $xy^iz \in L$ for every $i \geq 0$,
- 2 $|y| > 0$, and
- 3 $|xy| \leq p$.

Breaking it down:

- Condition 1: The same decomposition works for every pumping value $i \geq 0$.
- Condition 2: The loop consumes at least one symbol, so pumping actually changes the string when $i \neq 1$.
- Condition 3: The loop lies within the first p symbols.

Why the Pumping Lemma Is True

Let a DFA for L have p states, and let $s \in L$ with $|s| \geq p$. During the first p input symbols, record the $p + 1$ visited states:

$$q_0, q_1, \dots, q_p.$$

Two visits must be equal; choose $0 \leq r < t \leq p$ with $q_r = q_t$. Split s as follows:

$$\underbrace{s_1 \cdots s_r}_x \underbrace{s_{r+1} \cdots s_t}_y \underbrace{s_{t+1} \cdots s_{|s|}}_z.$$

Then $|y| = t - r > 0$ and $|xy| = t \leq p$. Because reading y returns the DFA to the same state, the loop may be traversed any number of times; hence $xy^i z \in L$ for every $i \geq 0$.

Quantifiers and the Lemma's Limitation

The logical order is the heart of a correct nonregularity proof:

$$\exists p \forall s \exists (x, y, z) \forall i \geq 0.$$

To contradict the lemma, reverse the game after assuming regularity:

$$\forall p \exists s \forall (x, y, z) \exists i \geq 0 \text{ such that } xy^iz \notin L,$$

subject to $s = xyz$, $|s| \geq p$, $|y| > 0$, and $|xy| \leq p$.

Necessary, Not Sufficient

Every regular language satisfies the pumping lemma, but satisfying the lemma does not by itself prove that a language is regular. Finite languages satisfy it vacuously by choosing p larger than every string length.

Quantifiers and the Lemma's Limitation

The logical order is the heart of a correct nonregularity proof:

$$\exists p \forall s \exists (x, y, z) \forall i \geq 0.$$

To contradict the lemma, reverse the game after assuming regularity:

$$\forall p \exists s \forall (x, y, z) \exists i \geq 0 \text{ such that } xy^iz \notin L,$$

subject to $s = xyz$, $|s| \geq p$, $|y| > 0$, and $|xy| \leq p$.

Necessary, Not Sufficient

Every regular language satisfies the pumping lemma, but satisfying the lemma does not by itself prove that a language is regular. Finite languages satisfy it vacuously by choosing p larger than every string length.

The Adversarial Game

The pumping lemma is used **negatively** to prove that a language is not regular. Its quantifiers can be remembered as a four-step game.

How to play the Pumping Game

- 1 **Opponent:** Gives you the pumping length p promised by the assumption that the language is regular.
- 2 **You:** Pick a strategic string $s \in L$ such that $|s| \geq p$.
- 3 **Opponent:** Chooses any valid split $s = xyz$ with $|y| > 0$ and $|xy| \leq p$.
- 4 **You:** After seeing the split, choose $i \geq 0$ so that $xy^iz \notin L$.

If this strategy works for every valid split, the initial regularity assumption is false.

The Adversarial Game

The pumping lemma is used **negatively** to prove that a language is not regular. Its quantifiers can be remembered as a four-step game.

How to play the Pumping Game

- 1 **Opponent:** Gives you the pumping length p promised by the assumption that the language is regular.
- 2 **You:** Pick a strategic string $s \in L$ such that $|s| \geq p$.
- 3 **Opponent:** Chooses any valid split $s = xyz$ with $|y| > 0$ and $|xy| \leq p$.
- 4 **You:** After seeing the split, choose $i \geq 0$ so that $xy^iz \notin L$.

If this strategy works for every valid split, the initial regularity assumption is false.

The Adversarial Game

The pumping lemma is used **negatively** to prove that a language is not regular. Its quantifiers can be remembered as a four-step game.

How to play the Pumping Game

- 1 **Opponent:** Gives you the pumping length p promised by the assumption that the language is regular.
- 2 **You:** Pick a strategic string $s \in L$ such that $|s| \geq p$.
- 3 **Opponent:** Chooses any valid split $s = xyz$ with $|y| > 0$ and $|xy| \leq p$.
- 4 **You:** After seeing the split, choose $i \geq 0$ so that $xy^iz \notin L$.

If this strategy works for every valid split, the initial regularity assumption is false.

The Adversarial Game

The pumping lemma is used **negatively** to prove that a language is not regular. Its quantifiers can be remembered as a four-step game.

How to play the Pumping Game

- 1 **Opponent:** Gives you the pumping length p promised by the assumption that the language is regular.
- 2 **You:** Pick a strategic string $s \in L$ such that $|s| \geq p$.
- 3 **Opponent:** Chooses any valid split $s = xyz$ with $|y| > 0$ and $|xy| \leq p$.
- 4 **You:** After seeing the split, choose $i \geq 0$ so that $xy^iz \notin L$.

If this strategy works for every valid split, the initial regularity assumption is false.

The Adversarial Game

The pumping lemma is used **negatively** to prove that a language is not regular. Its quantifiers can be remembered as a four-step game.

How to play the Pumping Game

- 1 **Opponent:** Gives you the pumping length p promised by the assumption that the language is regular.
- 2 **You:** Pick a strategic string $s \in L$ such that $|s| \geq p$.
- 3 **Opponent:** Chooses any valid split $s = xyz$ with $|y| > 0$ and $|xy| \leq p$.
- 4 **You:** After seeing the split, choose $i \geq 0$ so that $xy^iz \notin L$.

If this strategy works for every valid split, the initial regularity assumption is false.

The Adversarial Game

The pumping lemma is used **negatively** to prove that a language is not regular. Its quantifiers can be remembered as a four-step game.

How to play the Pumping Game

- 1 **Opponent:** Gives you the pumping length p promised by the assumption that the language is regular.
- 2 **You:** Pick a strategic string $s \in L$ such that $|s| \geq p$.
- 3 **Opponent:** Chooses any valid split $s = xyz$ with $|y| > 0$ and $|xy| \leq p$.
- 4 **You:** After seeing the split, choose $i \geq 0$ so that $xy^iz \notin L$.

If this strategy works for every valid split, the initial regularity assumption is false.

Example 1: Equal Blocks

Goal: Prove $L = \{a^n b^n : n \geq 0\}$ is not regular.

① Assume L is regular. The opponent provides pumping length p .

② **We choose:** $s = a^p b^p$. Note that $s \in L$ and $|s| = 2p \geq p$.

③ Opponent divides $s = xyz$.

Crucial insight: Since $|xy| \leq p$, both x and y lie in the first block of a 's. Thus $y = a^k$ for some $1 \leq k \leq p$.

④ **We pump:** Choose $i = 2$. The new string is xy^2z .

- $s = xy^1z$ had p a 's and p b 's.
- Pumping adds k extra a 's.
- The new string $xy^2z = a^{p+k}b^p$.

Since $k > 0$, the numbers of a 's and b 's differ. Thus $xy^2z \notin L$, contradicting the pumping lemma. Therefore L is not regular.

Example 1: Equal Blocks

Goal: Prove $L = \{a^n b^n : n \geq 0\}$ is not regular.

① Assume L is regular. The opponent provides pumping length p .

② **We choose:** $s = a^p b^p$. Note that $s \in L$ and $|s| = 2p \geq p$.

③ Opponent divides $s = xyz$.

Crucial insight: Since $|xy| \leq p$, both x and y lie in the first block of a 's. Thus $y = a^k$ for some $1 \leq k \leq p$.

④ **We pump:** Choose $i = 2$. The new string is xy^2z .

- $s = xy^1z$ had p a 's and p b 's.
- Pumping adds k extra a 's.
- The new string $xy^2z = a^{p+k}b^p$.

Since $k > 0$, the numbers of a 's and b 's differ. Thus $xy^2z \notin L$, contradicting the pumping lemma. Therefore L is not regular.

Example 1: Equal Blocks

Goal: Prove $L = \{a^n b^n : n \geq 0\}$ is not regular.

① Assume L is regular. The opponent provides pumping length p .

② **We choose:** $s = a^p b^p$. Note that $s \in L$ and $|s| = 2p \geq p$.

③ Opponent divides $s = xyz$.

Crucial insight: Since $|xy| \leq p$, both x and y lie in the first block of a 's. Thus $y = a^k$ for some $1 \leq k \leq p$.

④ **We pump:** Choose $i = 2$. The new string is xy^2z .

- $s = xy^1z$ had p a 's and p b 's.
- Pumping adds k extra a 's.
- The new string $xy^2z = a^{p+k}b^p$.

Since $k > 0$, the numbers of a 's and b 's differ. Thus $xy^2z \notin L$, contradicting the pumping lemma. Therefore L is not regular.

Example 1: Equal Blocks

Goal: Prove $L = \{a^n b^n : n \geq 0\}$ is not regular.

① Assume L is regular. The opponent provides pumping length p .

② **We choose:** $s = a^p b^p$. Note that $s \in L$ and $|s| = 2p \geq p$.

③ Opponent divides $s = xyz$.

Crucial insight: Since $|xy| \leq p$, both x and y lie in the first block of a 's. Thus $y = a^k$ for some $1 \leq k \leq p$.

④ **We pump:** Choose $i = 2$. The new string is xy^2z .

- $s = xy^1z$ had p a 's and p b 's.
- Pumping adds k extra a 's.
- The new string $xy^2z = a^{p+k}b^p$.

Since $k > 0$, the numbers of a 's and b 's differ. Thus $xy^2z \notin L$, contradicting the pumping lemma. Therefore L is not regular.

Example 1: Equal Blocks

Goal: Prove $L = \{a^n b^n : n \geq 0\}$ is not regular.

① Assume L is regular. The opponent provides pumping length p .

② **We choose:** $s = a^p b^p$. Note that $s \in L$ and $|s| = 2p \geq p$.

③ Opponent divides $s = xyz$.

Crucial insight: Since $|xy| \leq p$, both x and y lie in the first block of a 's. Thus $y = a^k$ for some $1 \leq k \leq p$.

④ **We pump:** Choose $i = 2$. The new string is xy^2z .

- $s = xy^1z$ had p a 's and p b 's.
- Pumping adds k extra a 's.
- The new string $xy^2z = a^{p+k}b^p$.

Since $k > 0$, the numbers of a 's and b 's differ. Thus $xy^2z \notin L$, contradicting the pumping lemma. Therefore L is not regular.

Example 1: Equal Blocks

Goal: Prove $L = \{a^n b^n : n \geq 0\}$ is not regular.

① Assume L is regular. The opponent provides pumping length p .

② **We choose:** $s = a^p b^p$. Note that $s \in L$ and $|s| = 2p \geq p$.

③ Opponent divides $s = xyz$.

Crucial insight: Since $|xy| \leq p$, both x and y lie in the first block of a 's. Thus $y = a^k$ for some $1 \leq k \leq p$.

④ **We pump:** Choose $i = 2$. The new string is xy^2z .

- $s = xy^1z$ had p a 's and p b 's.
- Pumping adds k extra a 's.
- The new string $xy^2z = a^{p+k}b^p$.

Since $k > 0$, the numbers of a 's and b 's differ. Thus $xy^2z \notin L$, contradicting the pumping lemma. Therefore L is not regular.

Example 2: Duplicated Strings

Goal: Prove $F = \{ww : w \in \{0,1\}^*\}$ is not regular.

- ① Assume F is regular. Opponent provides pumping length p .
- ② **We choose:** $s = 0^p 1 0^p 1 = (0^p 1)(0^p 1)$.
- ③ Because $|xy| \leq p$, a valid split has $y = 0^k$ for some $1 \leq k \leq p$.
- ④ **We pump:** Choose $i = 2$. The new string is xy^2z .
 - The first block of 0's now has $p + k$ zeros.
 - The new string is $0^{p+k} 1 0^p 1$.

The pumped string has exactly two 1's, separated by $p + 1$ positions. If it were uu , the distance between these two corresponding 1's would equal half the total length, $p + 1 + k/2$, which is impossible for $k > 0$ (and if k is odd, the total length is odd). Hence $xy^2z \notin F$, a contradiction.

Example 2: Duplicated Strings

Goal: Prove $F = \{ww : w \in \{0,1\}^*\}$ is not regular.

- ① Assume F is regular. Opponent provides pumping length p .
- ② We choose: $s = 0^p 1 0^p 1 = (0^p 1)(0^p 1)$.
- ③ Because $|xy| \leq p$, a valid split has $y = 0^k$ for some $1 \leq k \leq p$.
- ④ We pump: Choose $i = 2$. The new string is xy^2z .
 - The first block of 0's now has $p + k$ zeros.
 - The new string is $0^{p+k} 1 0^p 1$.

The pumped string has exactly two 1's, separated by $p + 1$ positions. If it were uu , the distance between these two corresponding 1's would equal half the total length, $p + 1 + k/2$, which is impossible for $k > 0$ (and if k is odd, the total length is odd). Hence $xy^2z \notin F$, a contradiction.

Example 2: Duplicated Strings

Goal: Prove $F = \{ww : w \in \{0,1\}^*\}$ is not regular.

- ① Assume F is regular. Opponent provides pumping length p .
- ② **We choose:** $s = 0^p 1 0^p 1 = (0^p 1)(0^p 1)$.
- ③ Because $|xy| \leq p$, a valid split has $y = 0^k$ for some $1 \leq k \leq p$.
- ④ **We pump:** Choose $i = 2$. The new string is xy^2z .
 - The first block of 0's now has $p + k$ zeros.
 - The new string is $0^{p+k} 1 0^p 1$.

The pumped string has exactly two 1's, separated by $p + 1$ positions. If it were uu , the distance between these two corresponding 1's would equal half the total length, $p + 1 + k/2$, which is impossible for $k > 0$ (and if k is odd, the total length is odd). Hence $xy^2z \notin F$, a contradiction.

Example 2: Duplicated Strings

Goal: Prove $F = \{ww : w \in \{0,1\}^*\}$ is not regular.

- ① Assume F is regular. Opponent provides pumping length p .
- ② **We choose:** $s = 0^p 1 0^p 1 = (0^p 1)(0^p 1)$.
- ③ Because $|xy| \leq p$, a valid split has $y = 0^k$ for some $1 \leq k \leq p$.
- ④ **We pump:** Choose $i = 2$. The new string is xy^2z .
 - The first block of 0's now has $p + k$ zeros.
 - The new string is $0^{p+k} 1 0^p 1$.

The pumped string has exactly two 1's, separated by $p + 1$ positions. If it were uu , the distance between these two corresponding 1's would equal half the total length, $p + 1 + k/2$, which is impossible for $k > 0$ (and if k is odd, the total length is odd). Hence $xy^2z \notin F$, a contradiction.

Example 2: Duplicated Strings

Goal: Prove $F = \{ww : w \in \{0,1\}^*\}$ is not regular.

- ① Assume F is regular. Opponent provides pumping length p .
- ② **We choose:** $s = 0^p 1 0^p 1 = (0^p 1)(0^p 1)$.
- ③ Because $|xy| \leq p$, a valid split has $y = 0^k$ for some $1 \leq k \leq p$.
- ④ **We pump:** Choose $i = 2$. The new string is xy^2z .
 - The first block of 0's now has $p + k$ zeros.
 - The new string is $0^{p+k} 1 0^p 1$.

The pumped string has exactly two 1's, separated by $p + 1$ positions. If it were uu , the distance between these two corresponding 1's would equal half the total length, $p + 1 + k/2$, which is impossible for $k > 0$ (and if k is odd, the total length is odd). Hence $xy^2z \notin F$, a contradiction.

Example 2: Duplicated Strings

Goal: Prove $F = \{ww : w \in \{0,1\}^*\}$ is not regular.

- ① Assume F is regular. Opponent provides pumping length p .
- ② **We choose:** $s = 0^p 1 0^p 1 = (0^p 1)(0^p 1)$.
- ③ Because $|xy| \leq p$, a valid split has $y = 0^k$ for some $1 \leq k \leq p$.
- ④ **We pump:** Choose $i = 2$. The new string is xy^2z .
 - The first block of 0's now has $p + k$ zeros.
 - The new string is $0^{p+k} 1 0^p 1$.

The pumped string has exactly two 1's, separated by $p + 1$ positions. If it were uu , the distance between these two corresponding 1's would equal half the total length, $p + 1 + k/2$, which is impossible for $k > 0$ (and if k is odd, the total length is odd). Hence $xy^2z \notin F$, a contradiction.

Example 3: Unary Perfect Squares

Goal: Prove $L = \{a^{n^2} : n \geq 0\}$ is not regular.

Pumped string length: $p^2 + |y|$



Since $|y| \leq p$, the pumped length is at most $p^2 + p$

The next square is $(p+1)^2 = p^2 + 2p + 1$

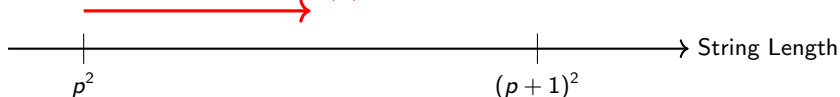
- 1 Pick $s = a^{p^2}$. Opponent divides $s = xyz$.
- 2 We pump $i = 2$. The length of xy^2z is $p^2 + |y|$.
- 3 Since $|y| \leq p$, then $p^2 < p^2 + |y| \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2$.

The pumped length lies strictly between consecutive squares, so $xy^2z \notin L$. This contradicts the pumping lemma.

Example 3: Unary Perfect Squares

Goal: Prove $L = \{a^{n^2} : n \geq 0\}$ is not regular.

Pumped string length: $p^2 + |y|$



Since $|y| \leq p$, the pumped length is at most $p^2 + p$

The next square is $(p+1)^2 = p^2 + 2p + 1$

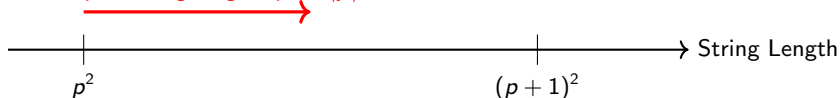
- 1 Pick $s = a^{p^2}$. Opponent divides $s = xyz$.
- 2 We pump $i = 2$. The length of xy^2z is $p^2 + |y|$.
- 3 Since $|y| \leq p$, then $p^2 < p^2 + |y| \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2$.

The pumped length lies strictly between consecutive squares, so $xy^2z \notin L$. This contradicts the pumping lemma.

Example 3: Unary Perfect Squares

Goal: Prove $L = \{a^{n^2} : n \geq 0\}$ is not regular.

Pumped string length: $p^2 + |y|$



Since $|y| \leq p$, the pumped length is at most $p^2 + p$

The next square is $(p+1)^2 = p^2 + 2p + 1$

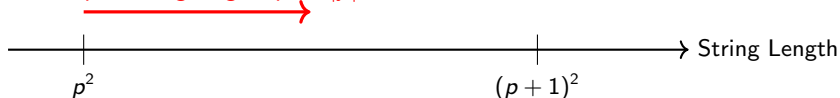
- 1 Pick $s = a^{p^2}$. Opponent divides $s = xyz$.
- 2 We pump $i = 2$. The length of xy^2z is $p^2 + |y|$.
- 3 Since $|y| \leq p$, then $p^2 < p^2 + |y| \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2$.

The pumped length lies strictly between consecutive squares, so $xy^2z \notin L$. This contradicts the pumping lemma.

Example 3: Unary Perfect Squares

Goal: Prove $L = \{a^{n^2} : n \geq 0\}$ is not regular.

Pumped string length: $p^2 + |y|$



Since $|y| \leq p$, the pumped length is at most $p^2 + p$

The next square is $(p+1)^2 = p^2 + 2p + 1$

- 1 Pick $s = a^{p^2}$. Opponent divides $s = xyz$.
- 2 We pump $i = 2$. The length of xy^2z is $p^2 + |y|$.
- 3 Since $|y| \leq p$, then $p^2 < p^2 + |y| \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2$.

The pumped length lies strictly between consecutive squares, so $xy^2z \notin L$. This contradicts the pumping lemma.

Using Closure to Prove Nonregularity

Closure theorems can be used contrapositively.

Reduction Pattern

To prove a language K nonregular:

- 1 assume K is regular;
- 2 apply an operation that preserves regularity; and
- 3 obtain a language already known to be nonregular.

Example

Let

$$K = \{w \in \{a, b\}^* : |w|_a = |w|_b\}.$$

If K were regular, then its intersection with the regular language a^*b^* would be regular. But

$$K \cap a^*b^* = \{a^n b^n : n \geq 0\},$$

contradicting Example 1. Hence K is not regular.

Using Closure to Prove Nonregularity

Closure theorems can be used contrapositively.

Reduction Pattern

To prove a language K nonregular:

- 1 assume K is regular;
- 2 apply an operation that preserves regularity; and
- 3 obtain a language already known to be nonregular.

Example

Let

$$K = \{w \in \{a, b\}^* : |w|_a = |w|_b\}.$$

If K were regular, then its intersection with the regular language a^*b^* would be regular. But

$$K \cap a^*b^* = \{a^n b^n : n \geq 0\},$$

contradicting Example 1. Hence K is not regular.

Checkpoint Exercises

- ➊ **Closure Property Trap:** Given that L_1 is regular, and the union $L_1 \cup L_2$ is regular, does it necessarily follow that L_2 is regular?
- ➋ **A Surprising Result:** Let D contain the strings over $\{0, 1\}$ having equally many occurrences of 01 and 10 as consecutive substrings. Is D regular?
- ➌ **Choosing s :** If trying to prove $L = \{a^n b^n \mid n \geq 0\}$ is not regular, what is wrong with picking $s = a^2 b^2$ if the opponent's $p = 5$?

Checkpoint Exercises

- 1 **Closure Property Trap:** Given that L_1 is regular, and the union $L_1 \cup L_2$ is regular, does it necessarily follow that L_2 is regular?
- 2 **A Surprising Result:** Let D contain the strings over $\{0, 1\}$ having equally many occurrences of 01 and 10 as consecutive substrings. Is D regular?
- 3 **Choosing s :** If trying to prove $L = \{a^n b^n \mid n \geq 0\}$ is not regular, what is wrong with picking $s = a^2 b^2$ if the opponent's $p = 5$?

Checkpoint Exercises

- 1 **Closure Property Trap:** Given that L_1 is regular, and the union $L_1 \cup L_2$ is regular, does it necessarily follow that L_2 is regular?
- 2 **A Surprising Result:** Let D contain the strings over $\{0, 1\}$ having equally many occurrences of 01 and 10 as consecutive substrings. Is D regular?
- 3 **Choosing s :** If trying to prove $L = \{a^n b^n \mid n \geq 0\}$ is not regular, what is wrong with picking $s = a^2 b^2$ if the opponent's $p = 5$?

Teacher's Corner: Common Pitfalls

Pitfall 1: Picking a specific p

Students sometimes begin by fixing $p = 5$. This is invalid: the pumping length is an unknown constant. The strategy must work for every $p \geq 1$.

Pitfall 2: Making assumptions about y

Even for $s = a^p b^p$, you may not assume $y = a$. The opponent may choose $y = a^k$ for any $1 \leq k \leq p$. The proof succeeds because pumping every such nonempty block changes only the number of a 's.

Also verify $s \in L$: for example, $(ab)^p \notin \{a^n b^n : n \geq 0\}$ when $p > 1$, so it cannot be used as the witness string for Example 1.

Teacher's Corner: Common Pitfalls

Pitfall 1: Picking a specific p

Students sometimes begin by fixing $p = 5$. This is invalid: the pumping length is an unknown constant. The strategy must work for every $p \geq 1$.

Pitfall 2: Making assumptions about y

Even for $s = a^p b^p$, you may not assume $y = a$. The opponent may choose $y = a^k$ for any $1 \leq k \leq p$. The proof succeeds because pumping every such nonempty block changes only the number of a 's.

Also verify $s \in L$: for example, $(ab)^p \notin \{a^n b^n : n \geq 0\}$ when $p > 1$, so it cannot be used as the witness string for Example 1.

Hints and Solutions

- **Exercise 1 (Closure Trap):** False. Let $L_1 = \Sigma^*$. Then $L_1 \cup L_2 = \Sigma^*$ for any L_2 , including a nonregular language.
- **Exercise 2 (01 and 10 substrings):** Yes. Each change $0 \rightarrow 1$ must eventually be balanced by a change $1 \rightarrow 0$, except for a possible net change determined by the endpoints. Thus the counts are equal exactly for ε or when the first and last symbols agree; a DFA need only remember the first and current symbols.
- **Exercise 3 (Choosing s):** The witness must satisfy $|s| \geq p$. If $p = 5$, then $|a^2b^2| = 4$, so the pumping lemma makes no promise about that string.

Pro-Tip: Pumping Down

Sometimes pumping up ($i = 2$) does not break the defining property. The lemma also requires $xy^0z = xz \in L$, so deleting y is often the cleaner route to a contradiction.

Hints and Solutions

- **Exercise 1 (Closure Trap):** False. Let $L_1 = \Sigma^*$. Then $L_1 \cup L_2 = \Sigma^*$ for any L_2 , including a nonregular language.
- **Exercise 2 (01 and 10 substrings):** Yes. Each change $0 \rightarrow 1$ must eventually be balanced by a change $1 \rightarrow 0$, except for a possible net change determined by the endpoints. Thus the counts are equal exactly for ε or when the first and last symbols agree; a DFA need only remember the first and current symbols.
- **Exercise 3 (Choosing s):** The witness must satisfy $|s| \geq p$. If $p = 5$, then $|a^2b^2| = 4$, so the pumping lemma makes no promise about that string.

Pro-Tip: Pumping Down

Sometimes pumping up ($i = 2$) does not break the defining property. The lemma also requires $xy^0z = xz \in L$, so deleting y is often the cleaner route to a contradiction.

Appendix Road Map

This appendix provides optional material for tutorials, examinations, and advanced self-study:

- ① five additional pumping-lemma proofs in adversarial-game form;
- ② constructive proofs for homomorphism and right quotient;
- ③ solved book-style challenges on regular-language properties; and
- ④ advanced results: Myhill–Nerode, residual languages, Arden's lemma, inverse homomorphisms, and state complexity.

Suggested use

Select only the examples that match the class level. The main lecture is complete without the appendix.

Pumping Game A: Binary Palindromes

Prove

$$L = \{w \in \{0,1\}^* : w = w^R\}$$

is not regular.

Adversarial solution

- ❶ **Opponent:** Supplies pumping length p .
- ❷ **You:** Choose $s = 0^p 1 0^p$, a palindrome.
- ❸ **Opponent:** Chooses $s = xyz$ with $|xy| \leq p$ and $|y| > 0$. Hence $y = 0^k$ for some $1 \leq k \leq p$.
- ❹ **You:** Pump down with $i = 0$:

$$xz = 0^{p-k} 1 0^p.$$

The two sides of the central 1 now have different lengths, so xz is not a palindrome. This contradicts the pumping lemma.

Pumping Game B: Unary Prime Lengths

Let

$$L = \{a^q : q \text{ is prime}\}.$$

Assume L is regular.

Adversarial solution

- ❶ **Opponent:** Supplies pumping length p .
- ❷ **You:** Choose a prime $q \geq p$ and set $s = a^q$.
- ❸ **Opponent:** Chooses $s = xyz$. Since the alphabet is unary, $y = a^k$ for some $1 \leq k \leq p$.
- ❹ **You:** Choose $i = q + 1$. The pumped length is

$$|xy^{q+1}z| = q + qk = q(k + 1).$$

Both factors exceed 1, so the new length is composite. Therefore the pumped string is not in L , a contradiction.

Pumping Game C: Powers of Two

Prove

$$L = \{a^{2^n} : n \geq 0\}$$

is not regular.

Adversarial solution

- ❶ **Opponent:** Supplies pumping length p .
- ❷ **You:** Choose m so that $2^m > p$ and let $s = a^{2^m}$.
- ❸ **Opponent:** Chooses $s = xyz$, where $y = a^k$ and $1 \leq k \leq p$.
- ❹ **You:** Pump up once, choosing $i = 2$. The new length is $2^m + k$.

Because

$$2^m < 2^m + k < 2^m + 2^m = 2^{m+1},$$

the pumped length lies strictly between consecutive powers of two.

Pumping Game D: More Zeros Than Ones

Let

$$L = \{0^n 1^m : n > m \geq 0\}.$$

Assume L is regular.

Adversarial solution

- ❶ **Opponent:** Supplies pumping length p .
- ❷ **You:** Choose $s = 0^p 1^{p-1} \in L$.
- ❸ **Opponent:** Chooses $s = xyz$ with $|xy| \leq p$; therefore $y = 0^k$ for some $1 \leq k \leq p$.
- ❹ **You:** Pump down:

$$xz = 0^{p-k} 1^{p-1}.$$

Since $p - k \leq p - 1$, the number of zeros is no longer strictly greater than the number of ones. Thus $xz \notin L$.

Pumping Game E: A Divisibility Constraint

Prove that

$$L = \{a^n b^m : n, m \geq 1 \text{ and } n \mid m\}$$

is not regular.

Adversarial solution

- ❶ **Opponent:** Supplies pumping length p .
- ❷ **You:** Choose $s = a^p b^p \in L$ because $p \mid p$.
- ❸ **Opponent:** Chooses $s = xyz$. The prefix restriction forces $y = a^k$ for some $1 \leq k \leq p$.
- ❹ **You:** Pump with $i = 2$, obtaining

$$xy^2z = a^{p+k}b^p.$$

A positive integer $p + k > p$ cannot divide the smaller positive integer p . Hence the pumped string is not in L .

Homomorphisms: Definition and Closure Theorem

A homomorphism $h : \Sigma^* \rightarrow \Gamma^*$ is determined by its values on symbols and extended by

$$h(\varepsilon) = \varepsilon, \quad h(a_1 \cdots a_n) = h(a_1) \cdots h(a_n).$$

Theorem

If L is regular, then its homomorphic image $h(L) = \{h(w) : w \in L\}$ is regular.

Automaton construction

Let an automaton M recognize L . For every transition $q \xrightarrow{a} r$:

- replace it by a path spelling $h(a)$, using fresh intermediate states when $|h(a)| > 1$;
- use an ε -transition when $h(a) = \varepsilon$.

Keep the original start and accepting states. The result is an ε -NFA.

A path labeled w in M becomes a path labeled $h(w)$ in the new automaton, proving that its language is exactly $h(L)$.

Worked Homomorphism Examples

Let $L = (01)^*$.

Non-erasing homomorphism

If $h(0) = ab$ and $h(1) = c$, then

$$h(L) = \{(abc)^n : n \geq 0\} = (abc)^*.$$

Every transition on 0 is replaced by a two-edge path labeled a, b , and every transition on 1 becomes a c -edge.

Erasing homomorphism

If $g(0) = ab$ and $g(1) = \varepsilon$, then

$$g(L) = (ab)^*.$$

The 1-transition becomes an ε -move.

Common distinction

Homomorphic images may merge distinct source words or erase symbols. Closure states that the resulting language remains regular; it does not claim that h is one-to-one.

Right Quotient: Constructive Closure Proof

The right quotient of L_1 by L_2 is

$$L_1/L_2 = \{x : \text{there exists } y \in L_2 \text{ with } xy \in L_1\}.$$

Theorem

If L_1 and L_2 are regular, then L_1/L_2 is regular.

Let $M_1 = (Q, \Sigma, \delta, q_0, F)$ recognize L_1 . Define

$$F' = \{q \in Q : \exists y \in L_2, \delta^*(q, y) \in F\}.$$

Use the DFA

$$M' = (Q, \Sigma, \delta, q_0, F').$$

After reading x , M' accepts precisely when some suffix $y \in L_2$ can take the current state to an old accepting state. Therefore

$$x \in L(M') \iff \exists y \in L_2 : xy \in L_1.$$

The set F' is computable by product-graph reachability using DFAs for L_1 and L_2 .

Worked Right-Quotient Examples

Removing one required suffix

Let $L_1 = (ab)^*$ and $L_2 = \{b\}$. Then

$$L_1/L_2 = \{x : xb \in (ab)^*\} = (ab)^*a.$$

For example, $aba \in L_1/L_2$ because $abab \in (ab)^*$.

A family of possible suffixes

Over $\Sigma = \{0, 1\}$, let $L_1 = \Sigma^*01$ and $L_2 = 1^*$. Then

$$L_1/L_2 = \Sigma^*0 \cup \Sigma^*01.$$

The empty suffix keeps every word already ending in 01; a nonempty suffix from 1^* can complete any word ending in 0.

Advanced remark

For a fixed regular L_1 , L_1/L_2 is regular even for arbitrary L_2 , because only a subset of the finitely many states becomes final. Regularity of L_2 makes that subset effectively computable.

Challenge 1: Prefixes, Suffixes, and Factors

For a language L , define

$$\text{Pref}(L) = \{x : \exists y, xy \in L\},$$

$$\text{Suff}(L) = \{y : \exists x, xy \in L\},$$

$$\text{Fact}(L) = \{z : \exists x, y, xzy \in L\}.$$

Claim

If L is regular, all three languages are regular.

Solution. Let M be a DFA for L .

- For prefixes, make accepting every state that can reach an old accepting state.
- For suffixes, use an NFA whose possible start states are all states reachable from the original start.
- For factors, combine both changes: start at any reachable state and accept at any state that can reach an old final state.

Each construction has finitely many states, proving regularity.

Challenge 2: Infinite Regular Subsets

Problem

Prove that every infinite regular language contains an infinite regular sublanguage.

Solution. Let p be a pumping length for an infinite regular language L . Choose $s \in L$ with $|s| \geq p$, and take the decomposition $s = xyz$ guaranteed by the pumping lemma. Then

$$R = \{xy^iz : i \geq 0\}$$

is a subset of L . Since $|y| > 0$, these strings have distinct lengths, so R is infinite. Moreover,

$$R = \{x\}\{y\}^*\{z\},$$

which is regular by closure under concatenation and star.

Lesson

An infinite regular language always contains an explicitly describable ultimately periodic family.

Challenge 3: A Short Witness of Infinitude

Problem

Let a DFA have n states. Prove that its language is infinite iff it accepts some word w satisfying

$$n \leq |w| < 2n.$$

Forward direction. Choose a shortest accepted word of length at least n . If its length were at least $2n$, a repeated state among the first $n + 1$ visits would identify a loop of length at most n . Removing that loop would leave an accepted word still of length at least n , contradicting minimality.

Reverse direction. Any accepted word of length at least n repeats a state along its path. Pumping the resulting nonempty loop yields infinitely many accepted words.

Algorithmic consequence

Finiteness can be tested by searching useful cycles, or equivalently by testing acceptance at lengths $n, \dots, 2n - 1$.

Challenge 4: How Long Is a Counterexample?

Problem

DFA M_1 has m states and DFA M_2 has n states. If their languages differ, show that some distinguishing string has length less than mn .

Solution. Build the product DFA for the symmetric difference. It has at most mn states and accepts exactly the strings on which the two machines disagree. If the languages differ, an accepting product state is reachable.

Choose a shortest path to such a state. It cannot repeat a product state; otherwise deleting the intervening cycle would give a shorter accepting path. A simple path through at most mn states has at most $mn - 1$ edges. Therefore a distinguishing string exists with

$$|w| \leq mn - 1.$$

Practical use

Breadth-first search in the product not only decides equivalence but returns a shortest counterexample.

Challenge 5: Counting 01 and 10

Let $N_{01}(w)$ and $N_{10}(w)$ count overlapping adjacent occurrences. Determine whether

$$D = \{w : N_{01}(w) = N_{10}(w)\}$$

is regular.

Key invariant

For every nonempty binary string,

$$N_{01}(w) - N_{10}(w) = \begin{cases} 1, & w \text{ starts with 0 and ends with 1,} \\ -1, & w \text{ starts with 1 and ends with 0,} \\ 0, & w \text{ has equal first and last symbols.} \end{cases}$$

Each change from 0 to 1 is canceled by a later change back, except for the net change between the endpoints. Hence

$$D = \{\varepsilon\} \cup \{0, 1\} \cup 0\Sigma^*0 \cup 1\Sigma^*1,$$

which is regular. A DFA need only remember the first and current symbols.

About the Book-Style Challenges

The preceding problems are newly phrased solutions based on recurring exercise themes in standard theory-of-computation texts:

- closure constructions and decision algorithms;
- pumping consequences for infinite regular languages;
- product automata and shortest distinguishing strings; and
- finding a finite invariant for a seemingly unbounded counting property.

Study method

Before reading a solution, identify the most useful representation: DFA graph, product automaton, regular expression, closure identity, or pumping loop. The representation usually reveals the proof.

Myhill–Nerode: The Exact Regularity Test

For a language $L \subseteq \Sigma^*$, define

$$x \equiv_L y \iff \text{for every } z \in \Sigma^*, xz \in L \iff yz \in L.$$

Thus x and y are equivalent when no future suffix can distinguish them.

Myhill–Nerode Theorem

L is regular if and only if $\equiv_L$ has finitely many equivalence classes. Moreover, the number of classes equals the number of states in the minimal DFA for L .

Interpretation

A DFA state represents all prefixes having the same possible futures. Minimization merges exactly the states whose prefixes cannot be distinguished by any continuation.

Myhill–Nerode: The Exact Regularity Test

For a language $L \subseteq \Sigma^*$, define

$$x \equiv_L y \iff \text{for every } z \in \Sigma^*, xz \in L \iff yz \in L.$$

Thus x and y are equivalent when no future suffix can distinguish them.

Myhill–Nerode Theorem

L is regular if and only if $\equiv_L$ has finitely many equivalence classes. Moreover, the number of classes equals the number of states in the minimal DFA for L .

Interpretation

A DFA state represents all prefixes having the same possible futures. Minimization merges exactly the states whose prefixes cannot be distinguished by any continuation.

Myhill–Nerode: The Exact Regularity Test

For a language $L \subseteq \Sigma^*$, define

$$x \equiv_L y \iff \text{for every } z \in \Sigma^*, xz \in L \iff yz \in L.$$

Thus x and y are equivalent when no future suffix can distinguish them.

Myhill–Nerode Theorem

L is regular if and only if $\equiv_L$ has finitely many equivalence classes. Moreover, the number of classes equals the number of states in the minimal DFA for L .

Interpretation

A DFA state represents all prefixes having the same possible futures. Minimization merges exactly the states whose prefixes cannot be distinguished by any continuation.

Myhill–Nerode Nonregularity Proof

Prove again that

$$L = \{a^n b^n : n \geq 0\}$$

is not regular. Consider the infinite collection of prefixes $\varepsilon, a, a^2, a^3, \dots$. If $i < j$, choose the distinguishing suffix $z = b^i$. Then

$$a^i z = a^i b^i \in L, \quad a^j z = a^j b^i \notin L.$$

Therefore $a^i \not\equiv_L a^j$ whenever $i \neq j$.

The relation $\equiv_L$ has infinitely many classes, so the Myhill–Nerode theorem implies that L is not regular.

Compared with pumping

The pumping lemma gives a necessary property of regular languages. Myhill–Nerode gives an exact characterization and also explains the state count of the minimal DFA.

Myhill–Nerode Nonregularity Proof

Prove again that

$$L = \{a^n b^n : n \geq 0\}$$

is not regular. Consider the infinite collection of prefixes $\varepsilon, a, a^2, a^3, \dots$. If $i < j$, choose the distinguishing suffix $z = b^i$. Then

$$a^i z = a^i b^i \in L, \quad a^j z = a^j b^i \notin L.$$

Therefore $a^i \not\equiv_L a^j$ whenever $i \neq j$.

The relation $\equiv_L$ has infinitely many classes, so the Myhill–Nerode theorem implies that L is not regular.

Compared with pumping

The pumping lemma gives a necessary property of regular languages. Myhill–Nerode gives an exact characterization and also explains the state count of the minimal DFA.

Myhill–Nerode Nonregularity Proof

Prove again that

$$L = \{a^n b^n : n \geq 0\}$$

is not regular. Consider the infinite collection of prefixes $\varepsilon, a, a^2, a^3, \dots$. If $i < j$, choose the distinguishing suffix $z = b^i$. Then

$$a^i z = a^i b^i \in L, \quad a^j z = a^j b^i \notin L.$$

Therefore $a^i \not\equiv_L a^j$ whenever $i \neq j$.

The relation $\equiv_L$ has infinitely many classes, so the Myhill–Nerode theorem implies that L is not regular.

Compared with pumping

The pumping lemma gives a necessary property of regular languages. Myhill–Nerode gives an exact characterization and also explains the state count of the minimal DFA.

Residual Languages and the Minimal DFA

The **left quotient** or **residual** of L by a word x is

$$x^{-1}L = \{z \in \Sigma^* : xz \in L\}.$$

It records all suffixes that can complete the prefix x into a word of L .

Structural facts

- $x \equiv_L y$ exactly when $x^{-1}L = y^{-1}L$.
- The distinct residuals of a regular language can serve as the states of its minimal DFA.
- The start state is $\varepsilon^{-1}L = L$; a residual is accepting exactly when it contains ε .
- Reading a moves $x^{-1}L$ to $(xa)^{-1}L$.

Hence the minimal DFA is unique up to renaming states: its states are the language's intrinsically different future behaviors.

Residual Languages and the Minimal DFA

The **left quotient** or **residual** of L by a word x is

$$x^{-1}L = \{z \in \Sigma^* : xz \in L\}.$$

It records all suffixes that can complete the prefix x into a word of L .

Structural facts

- $x \equiv_L y$ exactly when $x^{-1}L = y^{-1}L$.
- The distinct residuals of a regular language can serve as the states of its minimal DFA.
- The start state is $\varepsilon^{-1}L = L$; a residual is accepting exactly when it contains ε .
- Reading a moves $x^{-1}L$ to $(xa)^{-1}L$.

Hence the minimal DFA is unique up to renaming states: its states are the language's intrinsically different future behaviors.

Arden's Lemma: Solving Language Equations

Arden's Lemma

If $X, A, B \subseteq \Sigma^*$ satisfy

$$X = AX \cup B$$

and $\varepsilon \notin A$, then the unique solution is

$$X = A^*B.$$

Indeed, repeatedly substituting for X gives $X = B \cup AB \cup A^2B \cup \dots = A^*B$. The condition $\varepsilon \notin A$ guarantees uniqueness.

Example

The equation $X = aX \cup b$ has solution

$$X = a^*b.$$

Systems of such equations provide a systematic route from a finite automaton to a regular expression.

Arden's Lemma: Solving Language Equations

Arden's Lemma

If $X, A, B \subseteq \Sigma^*$ satisfy

$$X = AX \cup B$$

and $\varepsilon \notin A$, then the unique solution is

$$X = A^*B.$$

Indeed, repeatedly substituting for X gives $X = B \cup AB \cup A^2B \cup \dots = A^*B$. The condition $\varepsilon \notin A$ guarantees uniqueness.

Example

The equation $X = aX \cup b$ has solution

$$X = a^*b.$$

Systems of such equations provide a systematic route from a finite automaton to a regular expression.

Arden's Lemma: Solving Language Equations

Arden's Lemma

If $X, A, B \subseteq \Sigma^*$ satisfy

$$X = AX \cup B$$

and $\varepsilon \notin A$, then the unique solution is

$$X = A^*B.$$

Indeed, repeatedly substituting for X gives $X = B \cup AB \cup A^2B \cup \dots = A^*B$. The condition $\varepsilon \notin A$ guarantees uniqueness.

Example

The equation $X = aX \cup b$ has solution

$$X = a^*b.$$

Systems of such equations provide a systematic route from a finite automaton to a regular expression.

Inverse Homomorphisms Preserve Regularity

For a homomorphism $h : \Sigma^* \rightarrow \Gamma^*$ and $L \subseteq \Gamma^*$, define

$$h^{-1}(L) = \{w \in \Sigma^* : h(w) \in L\}.$$

Let $M = (Q, \Gamma, \delta, q_0, F)$ be a DFA for L . Construct a DFA over Σ with the same states, start state, and final states, but define

$$\delta'(q, a) = \delta^*(q, h(a)).$$

By induction on w , $\delta'^*(q_0, w) = \delta^*(q_0, h(w))$. Thus the new DFA accepts w exactly when $h(w) \in L$.

Example

If $h(0) = ab$, $h(1) = a$, and $L = (ab)^*$, then $h^{-1}(L) = 0^*$: every 0 contributes one ab , while any 1 leaves an unmatched a .

Inverse Homomorphisms Preserve Regularity

For a homomorphism $h : \Sigma^* \rightarrow \Gamma^*$ and $L \subseteq \Gamma^*$, define

$$h^{-1}(L) = \{w \in \Sigma^* : h(w) \in L\}.$$

Let $M = (Q, \Gamma, \delta, q_0, F)$ be a DFA for L . Construct a DFA over Σ with the same states, start state, and final states, but define

$$\delta'(q, a) = \delta^*(q, h(a)).$$

By induction on w , $\delta'^*(q_0, w) = \delta^*(q_0, h(w))$. Thus the new DFA accepts w exactly when $h(w) \in L$.

Example

If $h(0) = ab$, $h(1) = a$, and $L = (ab)^*$, then $h^{-1}(L) = 0^*$: every 0 contributes one ab , while any 1 leaves an unmatched a .

Inverse Homomorphisms Preserve Regularity

For a homomorphism $h : \Sigma^* \rightarrow \Gamma^*$ and $L \subseteq \Gamma^*$, define

$$h^{-1}(L) = \{w \in \Sigma^* : h(w) \in L\}.$$

Let $M = (Q, \Gamma, \delta, q_0, F)$ be a DFA for L . Construct a DFA over Σ with the same states, start state, and final states, but define

$$\delta'(q, a) = \delta^*(q, h(a)).$$

By induction on w , $\delta'^*(q_0, w) = \delta^*(q_0, h(w))$. Thus the new DFA accepts w exactly when $h(w) \in L$.

Example

If $h(0) = ab$, $h(1) = a$, and $L = (ab)^*$, then $h^{-1}(L) = 0^*$: every 0 contributes one ab , while any 1 leaves an unmatched a .

State Complexity: Construction Size Matters

Closure proves that a result is regular; **state complexity** asks how large a recognizing DFA may need to be.

Operation or conversion	General upper bound
complement of an n -state complete DFA	n states
union/intersection of m - and n -state DFAs	mn states
determinization of an n -state NFA	2^n subset states
reversal of an n -state DFA	at most 2^n DFA states

These are worst-case bounds: unreachable or equivalent constructed states may be removed, although suitable language families attain exponential or product-size growth.

Practical lesson

A mathematically simple closure construction can cause a substantial representation blow-up. Build reachable states on demand and minimize only when the application benefits from it.

State Complexity: Construction Size Matters

Closure proves that a result is regular; **state complexity** asks how large a recognizing DFA may need to be.

Operation or conversion	General upper bound
complement of an n -state complete DFA	n states
union/intersection of m - and n -state DFAs	mn states
determinization of an n -state NFA	2^n subset states
reversal of an n -state DFA	at most 2^n DFA states

These are worst-case bounds: unreachable or equivalent constructed states may be removed, although suitable language families attain exponential or product-size growth.

Practical lesson

A mathematically simple closure construction can cause a substantial representation blow-up. Build reachable states on demand and minimize only when the application benefits from it.

State Complexity: Construction Size Matters

Closure proves that a result is regular; **state complexity** asks how large a recognizing DFA may need to be.

Operation or conversion	General upper bound
complement of an n -state complete DFA	n states
union/intersection of m - and n -state DFAs	mn states
determinization of an n -state NFA	2^n subset states
reversal of an n -state DFA	at most 2^n DFA states

These are worst-case bounds: unreachable or equivalent constructed states may be removed, although suitable language families attain exponential or product-size growth.

Practical lesson

A mathematically simple closure construction can cause a substantial representation blow-up. Build reachable states on demand and minimize only when the application benefits from it.

Further Regular-Language Observations

- Every finite language is regular, and every infinite regular language contains an infinite regular sublanguage.
- A unary regular language has an *ultimately periodic* set of accepted lengths: after some point, membership repeats with a fixed period.
- Two DFA states are inequivalent exactly when some suffix is accepted from one state and rejected from the other; partition refinement finds these equivalence classes efficiently.
- The pumping length may be chosen as the number of states of a DFA, but it need not be the smallest possible pumping length for the language.
- Regular expressions, right-linear grammars, DFAs, and NFAs have the same expressive power, but conversions between them can change the representation size dramatically.

Further Regular-Language Observations

- Every finite language is regular, and every infinite regular language contains an infinite regular sublanguage.
- A unary regular language has an *ultimately periodic* set of accepted lengths: after some point, membership repeats with a fixed period.
- Two DFA states are inequivalent exactly when some suffix is accepted from one state and rejected from the other; partition refinement finds these equivalence classes efficiently.
- The pumping length may be chosen as the number of states of a DFA, but it need not be the smallest possible pumping length for the language.
- Regular expressions, right-linear grammars, DFAs, and NFAs have the same expressive power, but conversions between them can change the representation size dramatically.

Further Regular-Language Observations

- Every finite language is regular, and every infinite regular language contains an infinite regular sublanguage.
- A unary regular language has an *ultimately periodic* set of accepted lengths: after some point, membership repeats with a fixed period.
- Two DFA states are inequivalent exactly when some suffix is accepted from one state and rejected from the other; partition refinement finds these equivalence classes efficiently.
- The pumping length may be chosen as the number of states of a DFA, but it need not be the smallest possible pumping length for the language.
- Regular expressions, right-linear grammars, DFAs, and NFAs have the same expressive power, but conversions between them can change the representation size dramatically.

Further Regular-Language Observations

- Every finite language is regular, and every infinite regular language contains an infinite regular sublanguage.
- A unary regular language has an *ultimately periodic* set of accepted lengths: after some point, membership repeats with a fixed period.
- Two DFA states are inequivalent exactly when some suffix is accepted from one state and rejected from the other; partition refinement finds these equivalence classes efficiently.
- The pumping length may be chosen as the number of states of a DFA, but it need not be the smallest possible pumping length for the language.
- Regular expressions, right-linear grammars, DFAs, and NFAs have the same expressive power, but conversions between them can change the representation size dramatically.

Further Regular-Language Observations

- Every finite language is regular, and every infinite regular language contains an infinite regular sublanguage.
- A unary regular language has an *ultimately periodic* set of accepted lengths: after some point, membership repeats with a fixed period.
- Two DFA states are inequivalent exactly when some suffix is accepted from one state and rejected from the other; partition refinement finds these equivalence classes efficiently.
- The pumping length may be chosen as the number of states of a DFA, but it need not be the smallest possible pumping length for the language.
- Regular expressions, right-linear grammars, DFAs, and NFAs have the same expressive power, but conversions between them can change the representation size dramatically.

Appendix References and Further Practice



Peter Linz and Susan H. Rodger,

An Introduction to Formal Languages and Automata, 7th ed.,
Jones & Bartlett Learning, 2023.



Michael Sipser,

Introduction to the Theory of Computation, 3rd ed.,
Cengage Learning, 2013.



John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman,

Introduction to Automata Theory, Languages, and Computation, 3rd ed.,
Pearson, 2006.

The appendix problems are original formulations of standard exercise themes; consult these texts for further variations on closure, minimization, language equations, and nonregularity proofs.